



МИНОБРНАУКИ РОССИИ

**Федеральное государственное бюджетное образовательное учреждение
высшего образования**

**«МИРЭА – Российский технологический университет»
РТУ МИРЭА**

**Институт информационных технологий (ИТ)
Кафедра инструментального и прикладного программного обеспечения
(ИиППО)**

КУРСОВАЯ РАБОТА

по дисциплине: Фронтенд-разработка

по профилю: Разработка программных продуктов и проектирование информационных систем

направления профессиональной подготовки: 09.03.04 «Программная инженерия»

Тема: Онлайн-платформа для дистанционного обучения

Студент: Миркин Кирилл Леонидович

Группа: ИКБО-10-23

Руководитель: Мифтахов Эльдар Наилевич, д.ф.-м.н., профессор

Москва 2025



Цель и задачи

Цель работы:

разработка архитектуры системы управления онлайн-обучением с поддержкой тестирования и оценки знаний, основанной на современных принципах проектирования клиент-серверных приложений.

Задачи работы:

- выполнить анализ предметной области и определить сущности, роли и процессы, характерные для онлайн-обучения;
- сформулировать требования к системе, а также определить ограничения проекта;
- провести обзор архитектурных стилей и выполнить их сравнительный анализ;
- разработать набор архитектурных диаграмм;
- изучить и описать паттерны проектирования;
- обосновать выбор технологического стека;
- спроектировать и описать прототип.



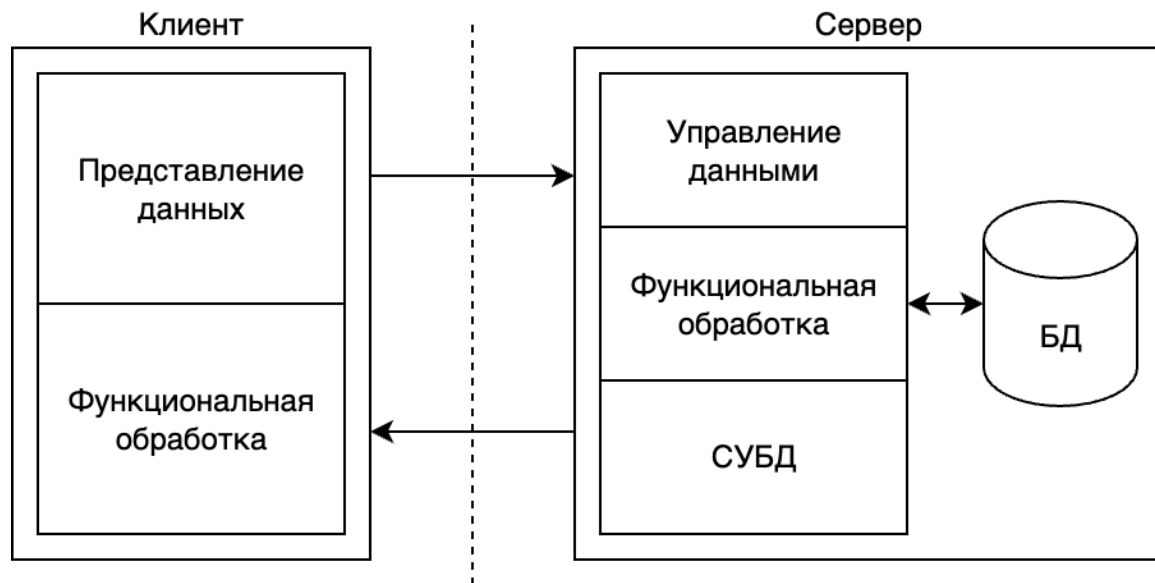
Описание предметной области

Предметная область разрабатываемой системы включает несколько ключевых компонентов:

- учебные курсы, содержащие теоретические материалы, практические задания, тесты и мультимедийные ресурсы;
- пользователей с различными ролями, среди которых администраторы, преподаватели и обучающиеся;
- механизмы тестирования и оценки знаний, обеспечивающие контроль усвоения материала;
- хранилище данных, в котором содержится информация о пользователях, курсах, результатах тестирования и связанных учебных артефактах.



Клиент-серверная архитектура



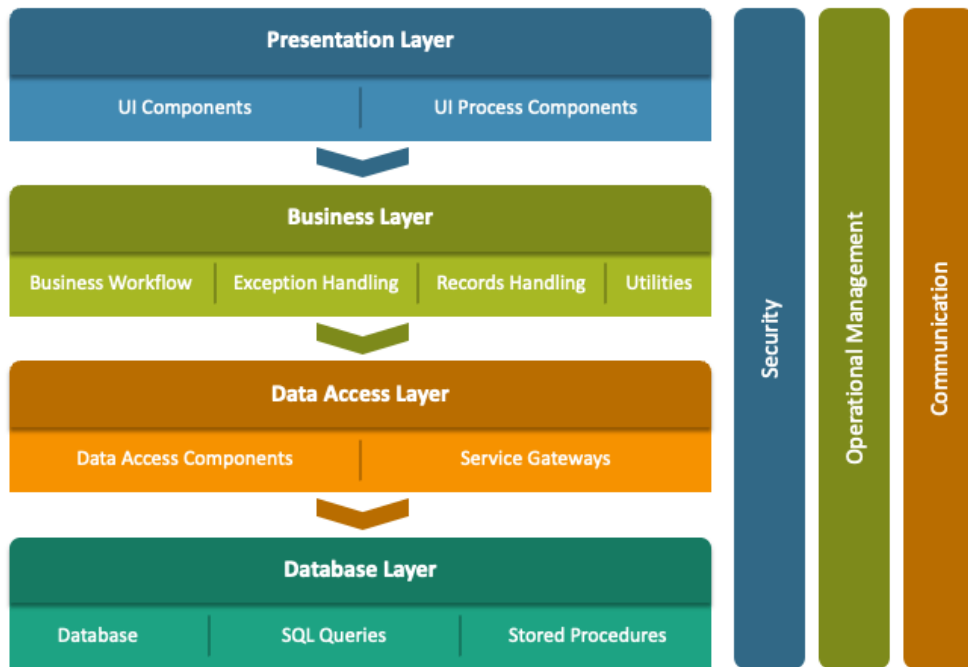
Клиент-серверная архитектура

Ключевые особенности клиент-серверной архитектуры:

- централизованное хранение данных на стороне сервера;
- разделение обязанностей между клиентом и сервером;
- предсказуемую модель обмена данными, основанную на запросах и ответах.



Многослойная архитектура



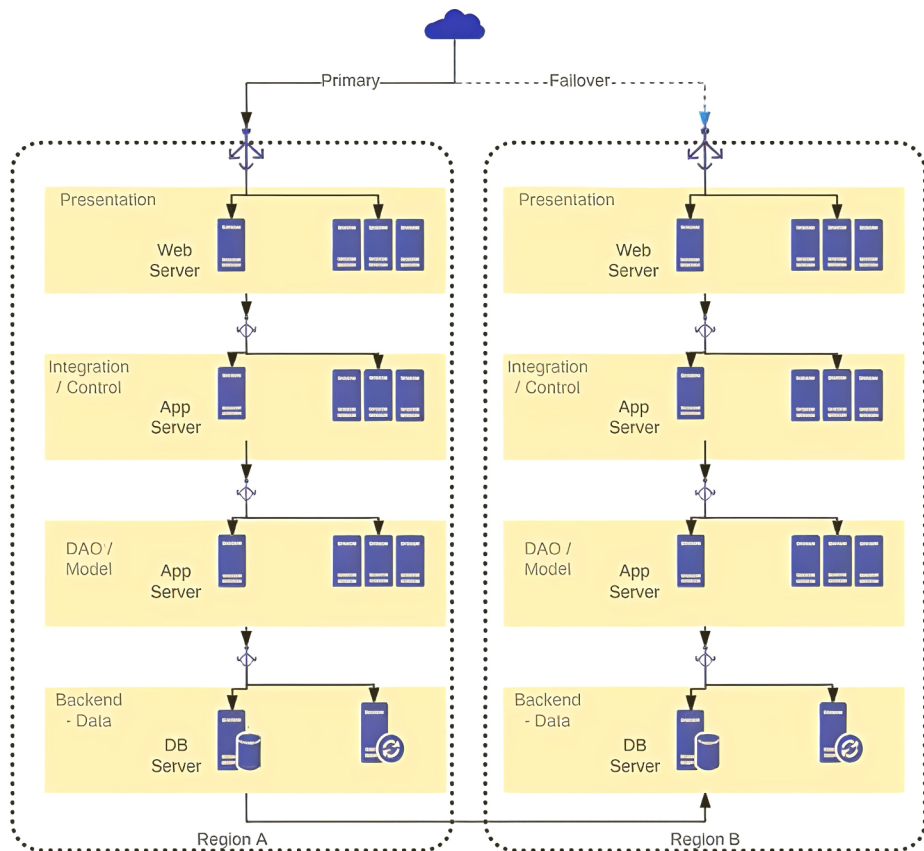
Многослойная архитектура

Классическая трёхслойная архитектура включает:

- слой представления;
- слой бизнес-логики;
- слой данных.



N-уровневая архитектура



N-уровневая архитектура

Наиболее распространённая трёхуровневая схема включает:

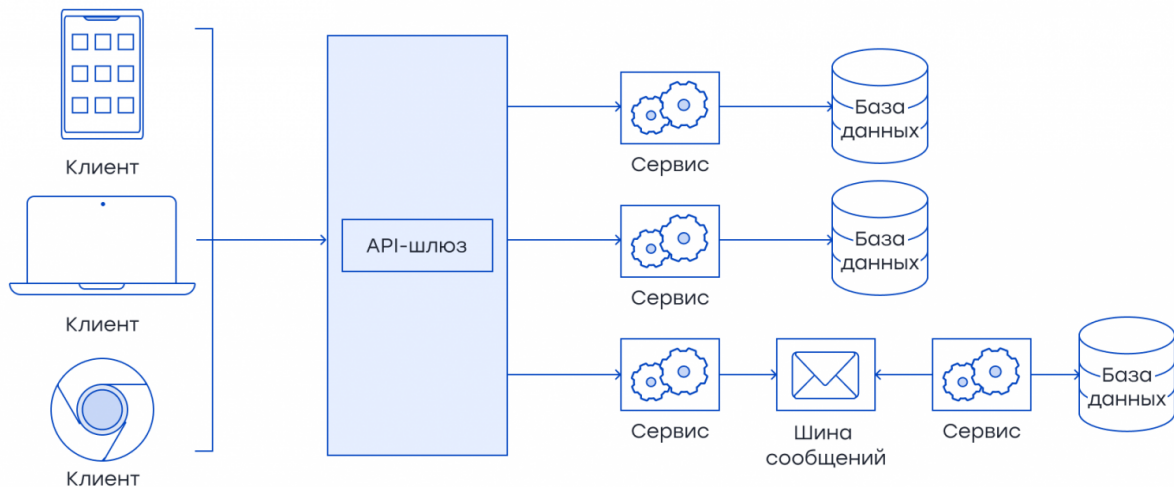
- уровень представления;
- уровень приложений;
- уровень хранения данных.



Микросервисная архитектура

Основные свойства
микросервисной архитектуры:

- автономность модулей;
- слабая связанность;
- масштабируемость отдельных функций.



Микросервисная архитектура



Сравнение архитектурных стилей

Таблица 1 — Сравнение архитектурных стилей

Характеристика	Клиент-серверная	Многослойная	N-уровневая	Микросервисная
Механизм разделения компонентов	Разделение на клиент и сервер	Логическое разделение на слои	Физическое разделение уровней	<u>Декомпозиция</u> на автономные сервисы
Степень связанности модулей	Высокая	Средняя	Средняя	Низкая
Масштабирование	Масштабирование сервера	Масштабирование слоёв	Масштабирование отдельных уровней	Масштабирование отдельных сервисов
Изменяемость и расширение	Изменения затрагивают сервер	Изменения изолируются в слое	Изменения возможны на уровне узла	Независимое развитие сервисов
Производственные требования	Минимальные	Умеренные	Повышенные	Высокие
Управление данными	Централизованная БД	Централизованная БД	БД на выделенном уровне	Раздельные хранилища или независимые БД



Диаграмма основных компонентов системы

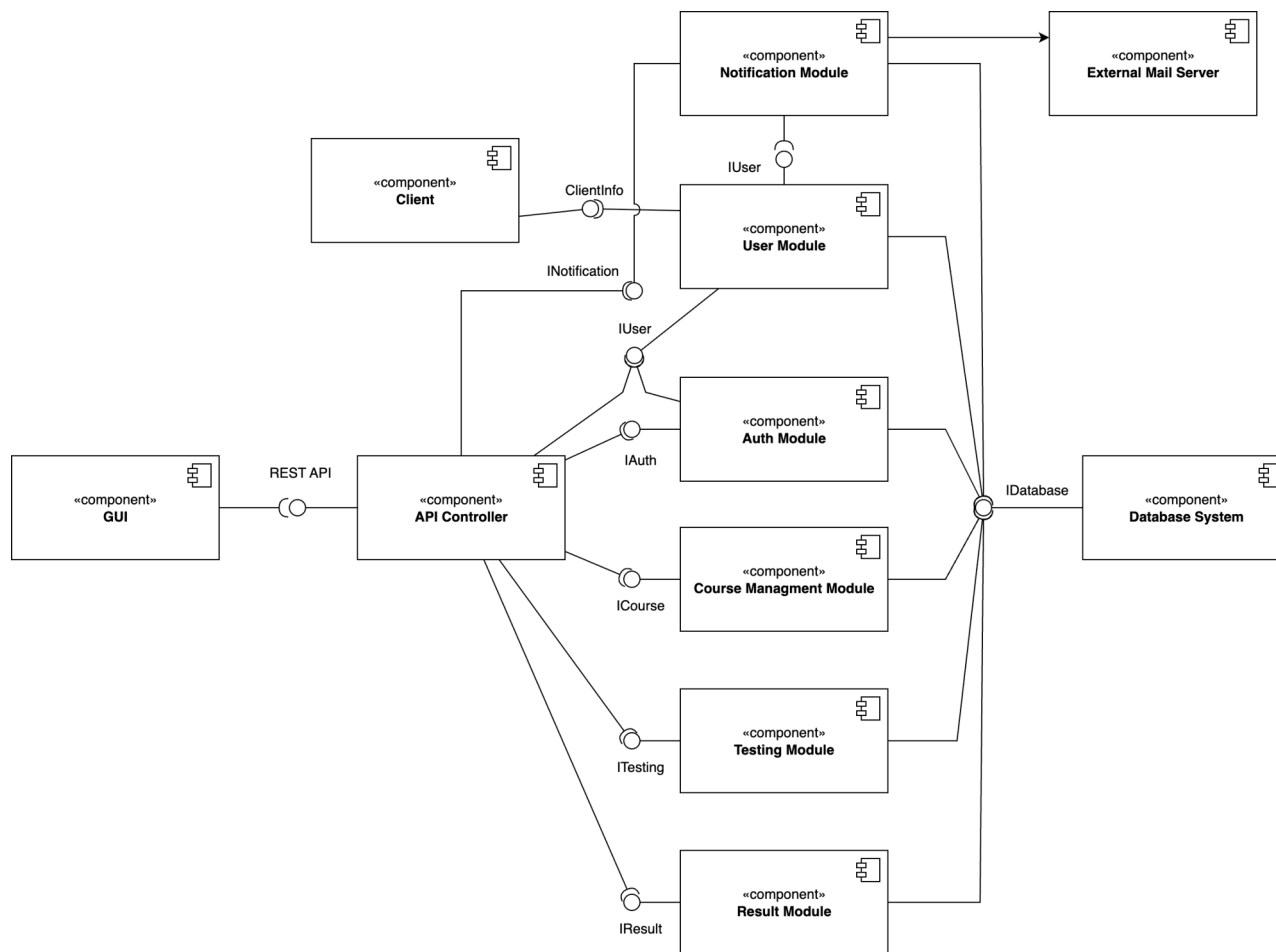
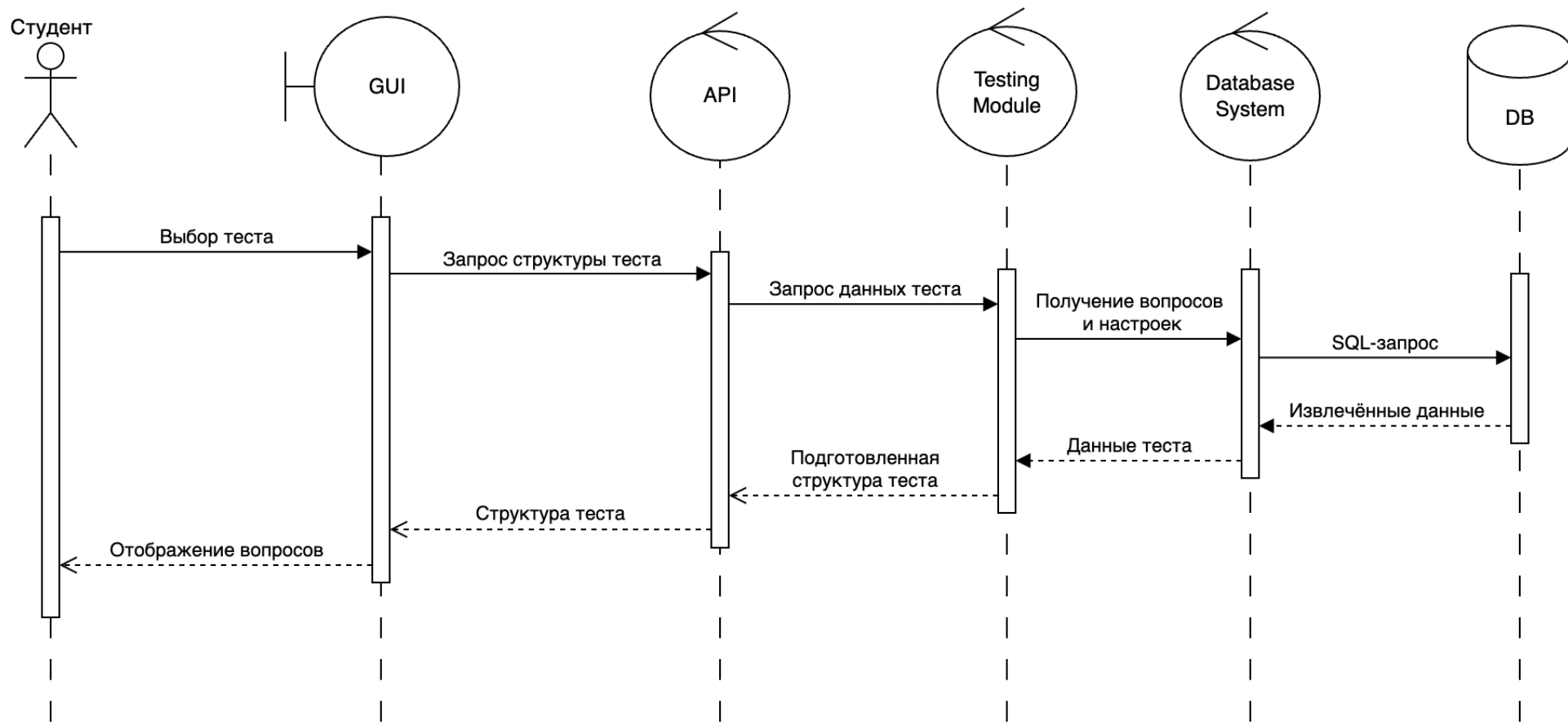




Диаграмма последовательности процесса получения теста



ER-диаграмма данных системы

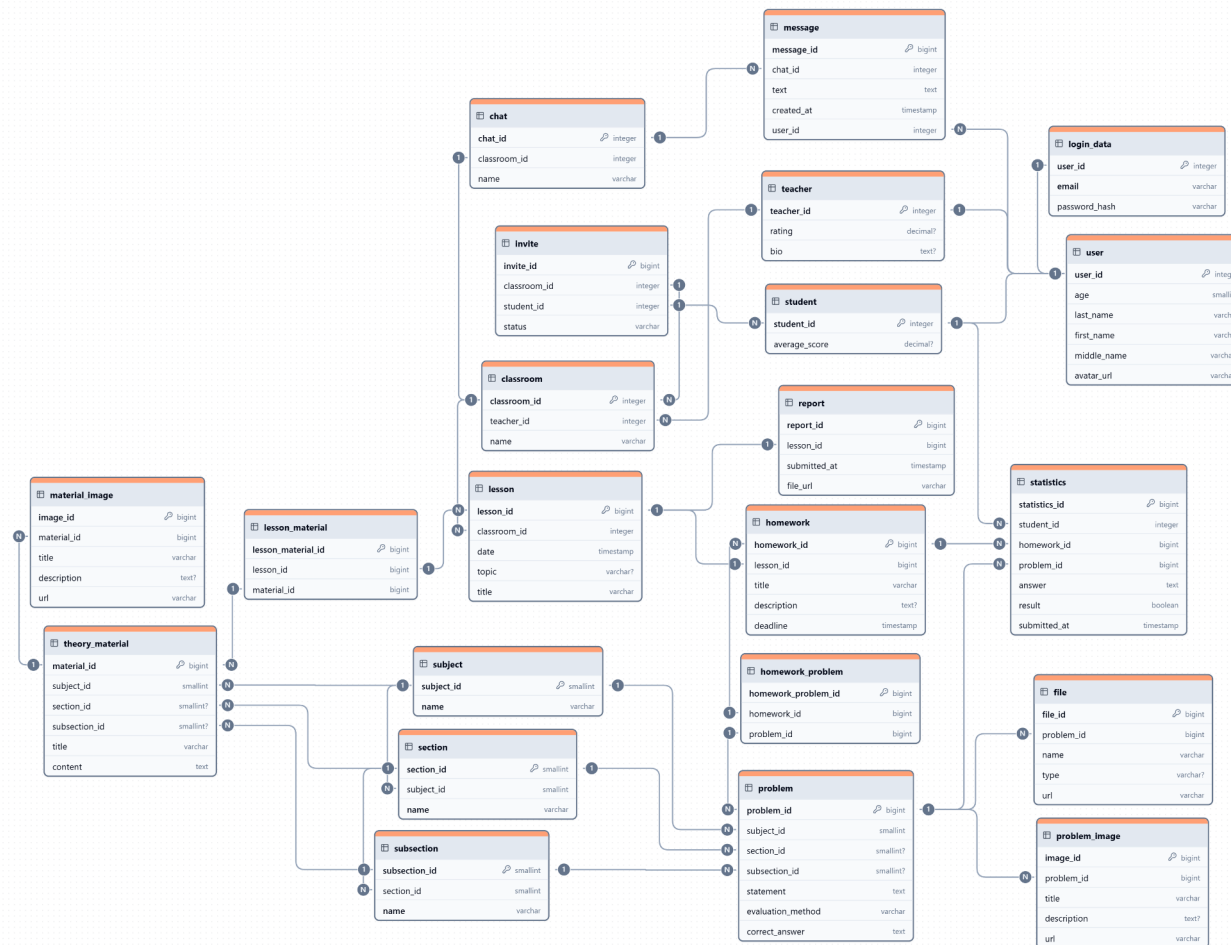
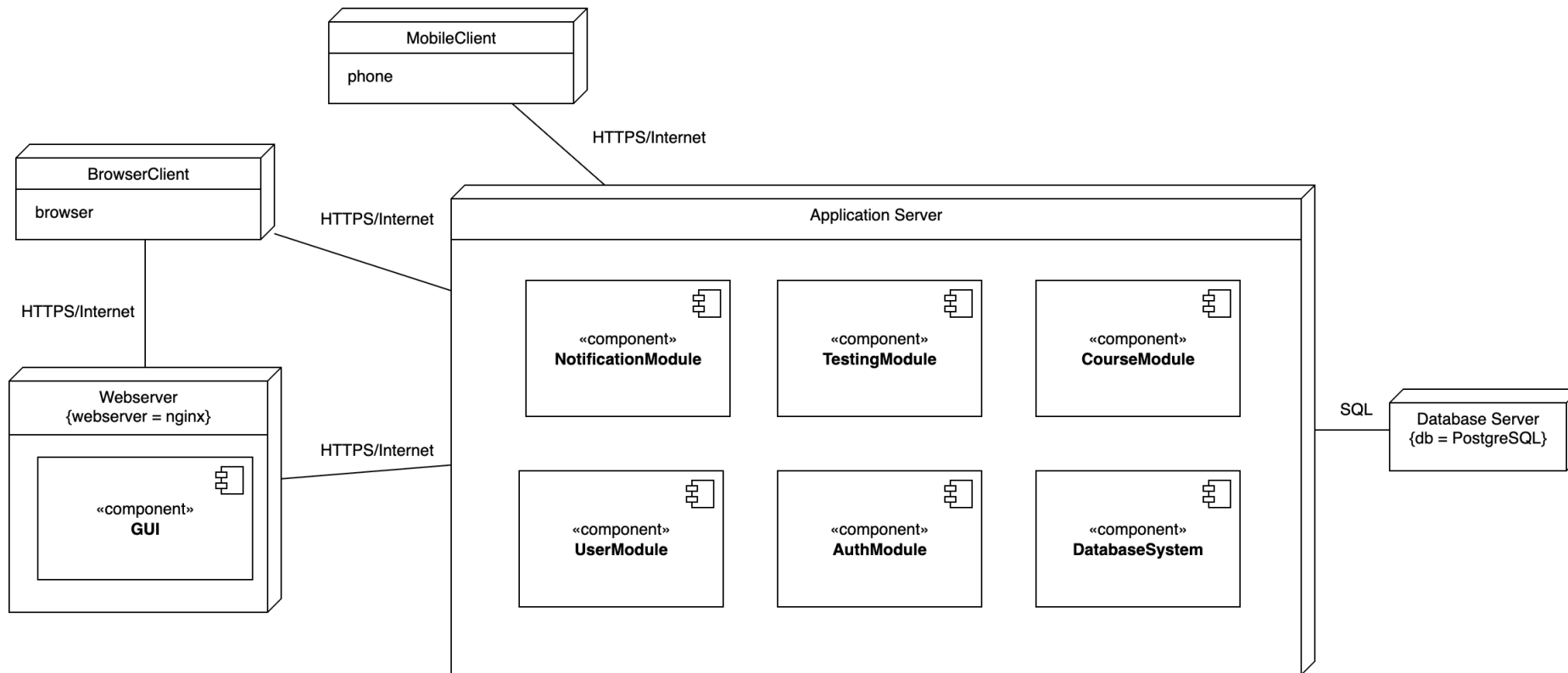
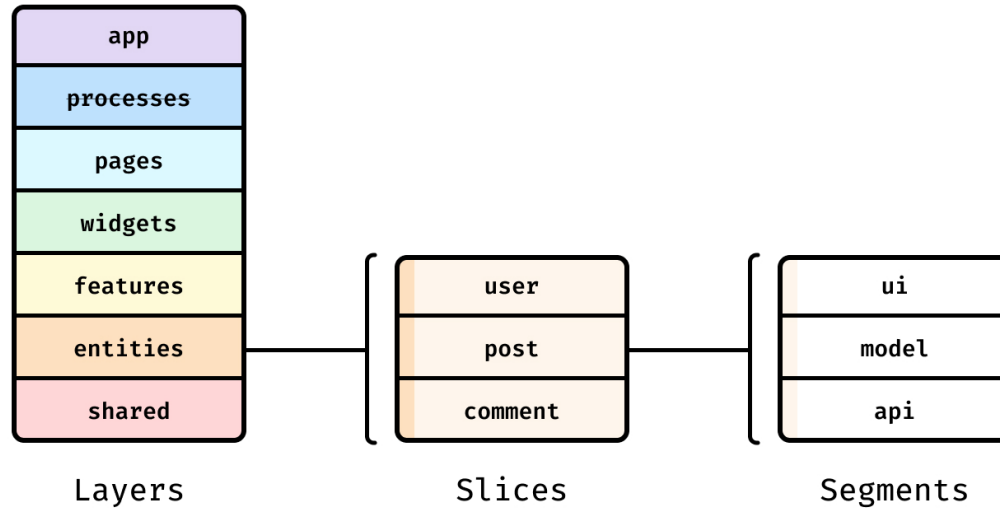




Диаграмма развёртывания



Архитектура серверной части проекта



Архитектура проекта на основе Feature-Sliced Design

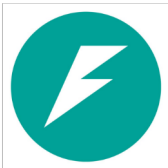
В проекте используется подход Feature-Sliced Design (FSD). Это упрощает масштабирование, переиспользование и тестирование.

Код разделён по слоям (shared, entities, features, pages, widgets) и слайсам, соответствующим предметной области.



Используемые технологии для серверной части

Для реализации серверной части платформы были выбраны следующие технологии:



– **FastAPI** — современный высокопроизводительный веб-фреймворк



– **Pydantic** — библиотека для валидации данных и работы с моделями

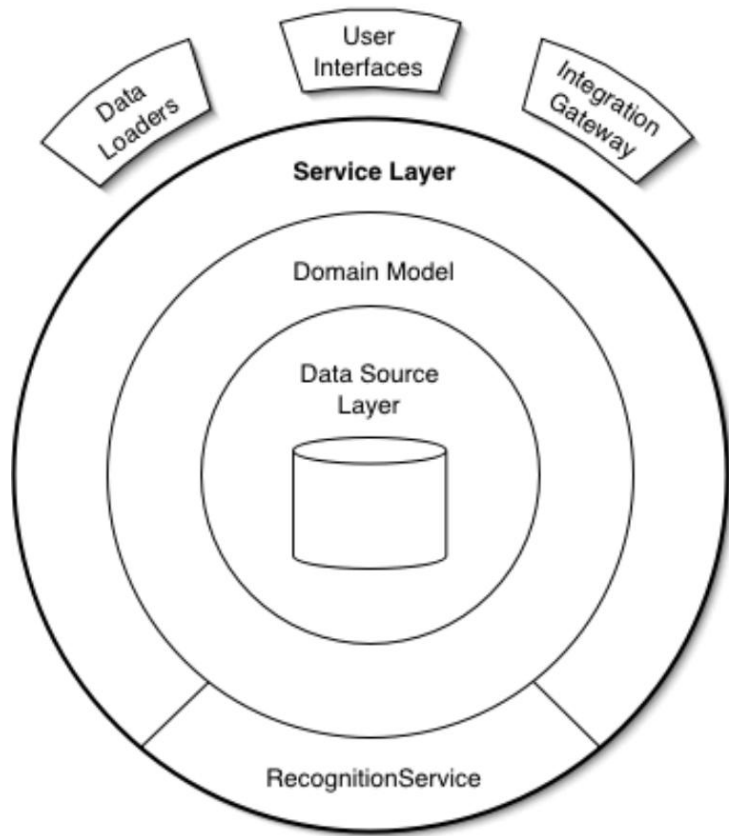


– **SQLAlchemy** — популярный инструмент объектно-реляционного отображения



– **Alembic** — система миграций базы данных для SQLAlchemy

Архитектура серверной части проекта



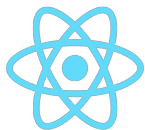
Паттерн разделения на слои

Архитектура сервера основана на паттернах разделения ответственности. Бизнес-логика организована в виде Service Layer, который централизует операции с пользователями, курсами и тестами. Data Mapper и Repository изолируют доступ к базе данных. Dependency Injection управляет зависимостями, снижая связанность и упрощая тестирование. Контроллеры обрабатывают HTTP-запросы, а DTO обеспечивают строгую валидацию данных. В совокупности это создаёт устойчивую, легко поддерживаемую основу.



Используемые технологии для клиентской части

Для реализации клиентской части платформы были выбраны следующие технологии:



– **React** — библиотека для создания пользовательского интерфейса



– **Vite** — инструмент сборки с мгновенной перезагрузкой



– **Tailwind CSS** — утилитарная система стилей



– **ShadCN UI** — библиотека компонентов на основе Radix UI



– **Zustand** — управление состоянием без лишней сложности



– **React Hook Form + Zod** — формы и валидация данных

Файловая структура клиентской части проекта



```
├── app
│   ├── app\providers
│   └── app\router
```

```
├── pages
│   ├── pages\auth
│   │   ├── pages\auth\login
│   │   │   └── pages\auth\login\ui
│   │   ├── pages\auth\register
│   │   │   └── pages\auth\register\ui
│   │   ├── pages\auth\reset-password
│   │   │   └── pages\auth\reset-password\ui
│   │   └── pages\auth\ui
│   ├── pages\notifications
│   │   └── pages\notifications\ui
│   ├── pages\profile
│   │   └── pages\profile\ui
│   ├── pages\student
│   │   ├── pages\student\class
│   │   │   └── pages\student\class\[id]
│   │   └── pages\student\ui
│   └── pages\teacher
│       ├── pages\teacher\class
│       │   └── pages\teacher\class\[id]
```

```
└── widgets\layout
```

```
├── features
│   ├── features\auth
│   │   ├── features\auth\api
│   │   ├── features\auth\lib
│   │   └── features\auth\model
│   │       └── features\auth\model\__tests__
│   ├── features\join-class
│   │   ├── features\join-class\api
│   │   ├── features\join-class\model
│   │   └── features\join-class\ui
│   ├── features\notifications
│   │   ├── features\notifications\api
│   │   ├── features\notifications\lib
│   │   ├── features\notifications\model
│   │   └── features\notifications\ui
│   └── features\profile
│       ├── features\profile\api
│       └── features\profile\model
```

```
├── shared
│   ├── shared\api
│   ├── shared\assets
│   │   └── shared\assets\icons
│   ├── shared\config
│   ├── shared\hooks
│   ├── shared\lib
│   ├── shared\styles
│   └── shared\ui
```

```
├── entities\class
│   ├── entities\class\lib
│   ├── entities\class\model
│   └── entities\class\ui
├── entities\many-to-many
├── entities\notification
│   ├── entities\notification\lib
│   ├── entities\notification\model
│   └── entities\notification\ui
│       └── entities\notification\ui\__test__
├── entities\student
│   ├── entities\student\lib
│   └── entities\student\model
├── entities\teacher
│   ├── entities\teacher\lib
│   └── entities\teacher\model
└── entities\user
    ├── entities\user\lib
    ├── entities\user\model
    │   └── entities\user\model\__test__
    └── entities\user\ui
```

Реализация компонентов



```
1 import { defineConfig } from 'vitest/config'
2 import react from '@vitejs/plugin-react'
3 import path from 'path'
4
5 export default defineConfig({
6   plugins: [react()],
7   resolve: {
8     alias: {
9       '@': path.resolve(__dirname, 'src'),
10     },
11   },
12   test: {
13     globals: true,
14     environment: 'jsdom',
15     setupFiles: ['./vitest.setup.ts'],
16     coverage: {
17       provider: 'v8',
18       reporter: ['text', 'html'],
19       include: ['src/entities/user', 'src/entities/notification'],
20       exclude: [
21         'src/**/*.test.*',
22         'src/**/*.stories.*',
23         'src/pages/**',
24         'src/widgets/**',
25         'src/shared/**',
26         'src/app/**',
27         'src/main.tsx',
28         'src/vite-env.d.ts',
29       ],
30     },
31   },
32 })
```

Содержимое vite.config.ts

```
1 {
2   "compilerOptions": {
3     "target": "ESNext",
4     "useDefineForClassFields": true,
5     "lib": ["DOM", "DOM.Iterable", "ESNext"],
6     "allowJs": false,
7     "skipLibCheck": true,
8     "esModuleInterop": false,
9     "allowSyntheticDefaultImports": true,
10    "strict": true,
11    "forceConsistentCasingInFileNames": true,
12    "noFallthroughCasesInSwitch": true,
13    "module": "ESNext",
14    "moduleResolution": "Bundler",
15    "resolveJsonModule": true,
16    "isolatedModules": true,
17    "noEmit": true,
18    "jsx": "react-jsx",
19    "baseUrl": ".",
20    "paths": {
21      "@/*": ["src/*"]
22    },
23    "types": ["vitest", "@testing-library/jest-dom"]
24  },
25  "include": ["src"],
26  "exclude": ["node_modules", "dist"]
27 }
```

Содержимое tsconfig.json



Реализация компонентов

Файлы, связанные с пользователями (User)

```
UserAvatar.tsx
src > entities > user > ui > UserAvatar.tsx > ...
1 import { Avatar, AvatarFallback, AvatarImage } from "@shared/ui/avatar";
2 import { useUserStore } from "../model/store";
3
4 interface UserAvatarProps {
5   avatar?: string;
6   className?: string;
7   size?: string; // e.g. "h-10 w-10"
8 }
9
10 export function UserAvatar({ avatar, className = "", size = "h-10 w-10 " }: UserAvatarProps) {
11   const user = useUserStore((s) => s.user);
12
13   const initials = [user!.firstName, user!.lastName]
14     .map((n) => n?.[0])
15     .join("")
16     .toUpperCase();
17
18   return (
19     <Avatar className={` ${size} ${className}`}>
20       <AvatarImage
21         src={avatar ? avatar : user!.avatar}
22         alt={user!.firstName}
23         className="object-cover h-full w-full rounded-full"
24       />
25       <AvatarFallback>{initials}</AvatarFallback>
26     </Avatar>
27   );
28 }
29
TS formats
src > entities > user > lib > TS formats > ...
1 export function getFullNameAdaptive(
2   user: { lastName?: string; firstName?: string; middleName?: string },
3   showMiddle: boolean
4 ) {
5   const parts = [user.lastName, user.firstName]
6   if (showMiddle && user.middleName) {
7     parts.push(user.middleName)
8   }
9   return parts.filter(Boolean).join(" ")
10 }
11

TS types.ts
src > entities > user > model > TS types.ts > *User > BirthDate
1 export interface User {
2   id: number
3   firstName: string
4   lastName: string
5   middleName?: string
6   email: string
7   role: UserRole
8   gender?: Gender
9   avatar?: string
10  phone?: string
11  address?: string
12  birthDate?: string
13  about?: string
14  contacts?: {
15    telegram?: string
16    vk?: string
17  }
18 }
19
20 export type UserRole = "student" | "teacher"
21
22 export type Gender = "male" | "female"
23
TS store.ts
src > entities > user > model > TS store.ts > ...
1 import { create } from "zustand"
2 import type { User } from "../types"
3
4 interface UserState {
5   user: User | null
6   setUser: (user: User | null) => void
7 }
8
9 export const useUserStore = create<UserState>((set) => ({
10   user: null,
11   setUser: (user) => set({ user }),
12 })))
13
```

Реализация компонентов



ивную **Профиль**



[Препода](#)
Смирнов Иван Александрович



Смирнов Иван Александрович

Ученик

Преподаватель

Личная информация

Редактировать

Фамилия

Смирнов

Имя

Иван

Отчество

Александрович

Дата рождения

1982-03-15

Пол

Мужской

О себе

Преподаватель математики. Работаю в школе №123 более 10 лет.

Контакты

Email

teacher@school.edu

Телефон

+7 (999) 123-45-67

Адрес

г. Москва, ул. Примерная, д. 42

Telegram

@example_DNnviJDfui



Реализация компонентов

Файлы, связанные с аутентификацией (использование Zod и Zustand)

```
TS schemats X
src > features > auth > model > TS schemats > ...
1 import { z } from "zod"
2
3 export const loginSchema = z.object({
4   email: z.string().email("Введите корректный email"),
5   password: z.string().min(6, "Минимум 6 символов"),
6   rememberMe: z.boolean().optional(),
7 })
8
9 export type LoginData = z.infer<typeof loginSchema>
10
11 export const resetPasswordSchema = z.object({
12   email: z.string().email("Введите корректный email"),
13 })
14
15 export type ResetPasswordData = z.infer<typeof resetPasswordSchema>
16
17 export const registerSchema = z.object({
18   lastName: z.string().min(1, "Введите фамилию"),
19   firstName: z.string().min(1, "Введите имя"),
20   middleName: z.string().optional(),
21
22   email: z.string().email("Введите корректный email"),
23
24   password: z.string().min(6, "Пароль должен содержать минимум 6 символов"),
25   confirmPassword: z.string(),
26
27   dateOfBirth: z.string().optional(),
28
29   description: z.string().max(500, "Максимум 500 символов").optional(),
30
31   agreeTerms: z.boolean().refine((v) => v === true, {
32     message: "Необходимо согласие с условиями использования",
33   }),
34   role: z.enum(["student", "teacher"], {
35     required_error: "Выберите роль",
36   }),
37   gender: z.enum(["male", "female"], {
38     required_error: "Выберите пол",
39   }),
40 }).refine((data) => data.password === data.confirmPassword, {
41   path: ["confirmPassword"],
42   message: "Пароли не совпадают",
43 })

TS store.ts X
src > features > auth > model > TS store.ts > AuthState > bootstrapped
1 import { create } from "zustand"
2 import { authApi } from "@features/auth/api/api"
3 import type { LoginData, RegisterData } from "./schema"
4 import { useUserStore } from "@entities/user/model/store"
5
6 interface AuthState {
7   loading: boolean
8   error: string | null
9   bootstrapped: boolean
10
11   login: (data: LoginData) => Promise<void>
12   register: (data: RegisterData) => Promise<void>
13   logout: () => void
14   checkAuth: () => Promise<void>
15   setError: (message: string | null) => void
16   setBootstrapped: () => void
17 }
18
19 export const useAuthStore = create<AuthState>((set) => ({
20   loading: false,
21   error: null,
22   bootstrapped: false,
23
24   async login(data) {
25     set({ loading: true, error: null })
26     try {
27       const user = await authApi.login(data)
28       const storage = data.rememberMe ? localStorage : sessionStorage
29       storage.setItem("auth-user", JSON.stringify(user))
30       useUserStore.getState().setUser(user)
31     } catch (e: any) {
32       set({ error: e.message || "Ошибка авторизации" })
33       throw e
34     } finally {
35       set({ loading: false })
36     }
37   },
38
39   async register(data) {
40     set({ loading: true, error: null })
41     try {
42       const user = await authApi.register(data)
43       useUserStore.getState().setUser(user)
44     } catch (e: any) {
45       set({ error: e.message || "Ошибка регистрации" })
46       throw e
47     } finally {
48       set({ loading: false })
49     }
50   },
51 })
52
53 export type AuthStore = typeof useAuthStore
```

Реализация компонентов



Вход в систему

Войдите в свою учетную запись для доступа к образовательной платформе

Email

 hello@w

Введите корректный email

Пароль

 ..

Минимум 6 символов

☐ Запомнить меня

[Забыли пароль?](#)

Войти

Нет учетной записи? [Зарегистрироваться](#)

Реализация компонентов



Основной макет страницы

MainLayout.tsx

src > widgets > layout > MainLayout.tsx > ...

```
1 import { ResponsiveHeader } from "@widgets/header/ResponsiveHeader";
2 import { Footer } from "@widgets/footer/Footer";
3
4 interface MainLayoutProps {
5   title?: string;
6   back?: { label: string; to: string };
7   footer?: "full" | "compact";
8   children: React.ReactNode;
9 }
10
11 export function MainLayout({ title, back, footer = "compact", children }: MainLayoutProps) {
12   return (
13     <div className="min-h-screen bg-white sm:bg-gray-100 flex flex-col">
14       <ResponsiveHeader title={title} overrideBack={back} />
15       <main className="flex-1 w-full sm:max-w-7xl mx-auto px-0 sm:px-6 lg:px-8 py-4 sm:py-8">{children}</main>
16       <Footer mode={footer} />
17     </div>
18   );
19 }
```

Реализация компонентов



Мои классы



Текущие классы

+ Вступить в класс



Математика 10А

Смирнов Иван Александрович

Активные задания

3

Перейти к классу

О платформе

WEP — современная образовательная платформа для эффективного взаимодействия учителей и учеников.

Контакты

✉ support@wep-edu.ru

☎ +7 (999) 123-45-67

📍 г. Москва, ул. Образовательная, 42

Разработчик

Миркин Кирилл Леонидович

🐙 GitHub: Kirill555dg

Реализация компонентов



Роутинг платформы

```
16 function App() {
17   return (
18     <>
19       <Toaster />
20       <AuthBootstrapper />
21       <ClassBootstrapper />
22
23       <Routes>
24         <Route path="/login" element={<LoginPage />} />
25         <Route path="/register" element={<RegisterPage />} />
26         <Route path="/reset-password" element={<ResetPasswordPage />} />
27
28         <Route element={<AuthProtectedRoute />>
29           <Route path="/profile" element={<ProfilePage />} />
30           <Route path="/notifications" element={<NotificationsPage />} />
31
32           <Route element={<RoleProtectedRoute requiredRole="student" />>
33             <Route path="/student" element={<StudentPage />} />
34           </Route>
35
36           <Route element={<RoleProtectedRoute requiredRole="teacher" />>
37             <Route path="/teacher" element={<TeacherPage />} />
38           </Route>
39
40           <Route path="/" element={<MainRedirect />} />
41         </Route>
42
43         <Route path="*" element={<Navigate to="/" replace />} />
44       </Routes>
45     </>
46   );
47 }
48
49 export default App;
```



Тестирование компонентов

Для юнит-тестирования использовался vitest (аналог jest), совместимый с vite

```
NotificationCard.test.tsx X
src > entities > notification > ui > __test__ > NotificationCard.test.tsx > ...
1 import { describe, it, expect, vi } from "vitest";
2 import { render, screen } from "@testing-library/react";
3 import userEvent from "@testing-library/user-event";
4 import { NotificationCard } from "../NotificationCard";
5 import { Notification } from "../../model/types";
6
7 const mockNotification: Notification = {
8   id: 1,
9   type: "homework_new",
10  title: "Новое задание",
11  description: "Сдайте уравнения",
12  date: "2025-05-11T14:30:00",
13  read: false,
14  link: "/homework/1",
15  linkText: "Перейти",
16  classTitle: "Математика 10А",
17 };
18
19 describe("NotificationCard", () => {
20   it("рендерит заголовок, описание и кнопку перехода", () => {
21     render(<NotificationCard notification={mockNotification} onNavigate={vi.fn()} />);
22
23     expect(screen.getByRole("heading", { name: /новое задание/i })).toBeInTheDocument();
24     expect(screen.getByText(/сдайте уравнения/i)).toBeInTheDocument();
25     expect(screen.getByRole("button", { name: /перейти/i })).toBeInTheDocument();
26   });
27
28   it("отображает кнопку 'Прочитано', если уведомление не прочитано", () => {
29     render(<NotificationCard notification={mockNotification} onMarkRead={vi.fn()} />);
30
31     expect(screen.getByRole("button", { name: /прочитано/i })).toBeInTheDocument();
32   });
33
34   it("не отображает кнопку 'Прочитано', если уведомление уже прочитано", () => {
35     render(<NotificationCard notification={{ ...mockNotification, read: true }} />);
36
37     expect(screen.queryByRole("button", { name: /прочитано/i })).not.toBeInTheDocument();
38   });
39 }
```

```
PS C:\Users\Kiril\dev\wep\wep-fsd-template> npm run vitest --coverage
DEV v3.1.3 C:\Users\Kiril\dev\wep\wep-fsd-template
Coverage enabled with v8

✓ src/entities/user/model/__test__/store.test.ts (2 tests) 2ms
✓ src/entities/notification/ui/__test__/NotificationCard.test.tsx (5 tests) 213ms
✓ src/features/auth/model/__tests__/store.test.ts (4 tests) 2127ms
✓ authStore > входит с валидными данными 804ms
✓ authStore > регистрирует нового пользователя 1010ms
✓ authStore > подтверждает авторизацию через checkAuth 312ms

Test Files 3 passed (3)
Tests 11 passed (11)
Start at 12:08:23
Duration 3.16s (transform 105ms, setup 378ms, collect 416ms, tests 2.34s, environment 1.50s, prepare 244ms)

% Coverage report from v8
-----|-----|-----|-----|-----|-----
File                                         | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----
All files                                   | 74.28   | 54.83    | 56.25   | 74.28   |
notification/lib                           | 63.63   | 20       | 75      | 63.63   |
  formatDate.ts                           | 100     | 33.33    | 100     | 100     | 8-9
  getNotificationBadge.tsx                 | 48.27   | 16.66    | 100     | 48.27   | 13-16,19-22,25-28,31-34,37-40
  getNotificationIcon.tsx                  | 70.58   | 16.66    | 100     | 70.58   | 9,11,13,15,17
  index.ts                                 | 100     | 100      | 100     | 100     |
  refreshNotifications.ts                  | 33.33   | 100      | 0       | 33.33   | 4-7
notification/model                         | 75.86   | 100      | 20      | 75.86   |
  mock-notifications.ts                   | 100     | 100      | 100     | 100     |
  store.ts                                | 30      | 100      | 0       | 30      | 19-22,25-31,34-41,44-45
  types.ts                                | 0       | 0        | 0       | 0       |
notification/ui                             | 100     | 100      | 100     | 100     |
  NotificationCard.tsx                    | 100     | 100      | 100     | 100     |
user/lib                                    | 0       | 0        | 0       | 0       |
  format.ts                                | 0       | 0        | 0       | 0       | 1-10
user/model                                 | 100     | 100      | 100     | 100     |
  mock-users.ts                           | 100     | 100      | 100     | 100     |
  store.ts                                 | 100     | 100      | 100     | 100     |
  types.ts                                | 0       | 0        | 0       | 0       |
user/ui                                    | 0       | 0        | 0       | 0       |
  UserAvatar.tsx                          | 0       | 0        | 0       | 0       | 1-28
-----|-----|-----|-----|-----|-----
```



Тестирование компонентов

Для интеграционного (e2e) тестирования использовался cypress

```
profile.cys.ts x
cypress > e2e > profile.cy.ts > describe("Страница профиля") callback
1 describe("Страница профиля", () => {
2   beforeEach(() => {
3     cy.visit("/login")
4
5     cy.get('input[name="email"]').type("teacher@school.edu")
6     cy.get('input[name="password"]').type("123456")
7     cy.get("button[type=submit]").click()
8
9     cy.url().should("include", "/teacher")
10    cy.visit("/profile")
11  })
12
13  it("отображает базовую информацию профиля", () => {
14    cy.contains("Профиль")
15    cy.contains("Смирнов") // фамилия
16    cy.contains("teacher@school.edu") // email
17    cy.contains("Мужской")
18    cy.get("img").should("exist")
19    cy.contains("себе")
20    cy.contains("Контакты")
21  })
22
23  it("отображает корректные ссылки на Telegram и VK", () => {
24    cy.contains("Telegram").parent().within(() => {
25      cy.get("a").should("have.attr", "href").and("include", "t.me")
26    })
27    cy.contains("VK").parent().within(() => {
28      cy.get("a").should("have.attr", "href").and("include", "vk.com")
29    })
30  })
31
```

Тестирование компонентов



File Edit View Window Help Developer Tools

Specs

profile cy.js 00:29

Страница профиля

- ✓ отображает базовую информацию профиля
- ✓ отображает корректные ссылки на Telegram и VK
- ✓ переходит в режим редактирования и изменяет поля
- ✓ отменяет изменения при нажатии на 'Отмена'
- ✓ меняет дату рождения
- ✓ меняет пол
- ✓ переключает роль и делает редирект
- ✓ отображается корректно на мобильном экране
- ✓ загружает новый аватар

BEFORE EACH

1 visit /login

2 get input[name="email"]

3 -type teacher@school.edu

4 get input[name="password"]

5 -type 123456

6 get button[type=submit]

7 -click

8 url

9 -assert expected http://localhost:5173/teacher to include /teacher

(new url) http://localhost:5173/teacher

10 visit /profile

http://localhost:5173/profile 1000x660 89%

На главную Профиль

WEP

4

Преподаватель Смирнов Иван Александрович



Смирнов Иван Александрович

Ученик Преподаватель

Личная информация

Редактировать

Фамилия Смирнов

Имя Иван

Отчество Александрович

Дата рождения 1982-03-15

Пол Мужской

О себе Преподаватель математики. Работаю в школе №123 более 10 лет.

Контакты

Деплой проекта



Проект размещён на платформе Vercel — хостинге для фронтенд-приложений.

Основные преимущества:

- Поддержка Vite без настройки;
- Автоматический деплой при коммитах (CI/CD);
- Быстрая доставка через CDN;
- Интеграция с GitHub.

education-platform master

t to create the project and give it a name.

Project Name

ects Hobby / web-education-pl

Settings

ables

Deploy

Настройка деплоя

Деплой проекта



▲

/

kirill555dg's projects

Hobby

/

wep-education-platform

⌵

Overview

Deployments

Analytics

Speed Insights

Logs

Observability

Firewall

Storage

Flags

AI

Settings

Ship tickets →

Feedback

wep-education-platform

Repository

Usage

Domains

Visit ⌵

Production Deployment

Build Logs

Runtime Logs

Instant Rollback

Deployment

wep-education-platform-h1v5qhzfp-kirill555dgs-projects.vercel.app

Domains

wep-education-platform.vercel.app +2

Status

Created

Ready

2m ago by Kirill555dg

Source

master

7f64da3 fix tests

> Deployment Configuration

☒ Fluid Compute

☒ Deployment Protection

☒ Skew Protection

To update your Production Deployment, push to the master branch.

Deployments

Firewall 24h Enable Bot Protection

Firewall is active

Observability 6h

Edge Requests

5

Function Invocations

0

Error Rate

0%

Analytics

Track visitors and page views

Enable

28



Результаты выполнения курсовой работы

В ходе выполнения работы была успешно достигнута цель — разработка клиентской части онлайн-платформы дистанционного обучения.

Результаты выполнения работы:

- выполнен анализ предметной области и существующих решений;
- спроектирован пользовательский интерфейс в Figma для ролей «ученик» и «учитель»;
- реализован интерфейс с применением компонентной архитектуры FSD;
- настроено управление состоянием с помощью Zustand;
- реализована маршрутизация с защитой доступа;
- выполнено модульное и e2e-тестирование (Vitest, Cypress);
- проект развёрнут на платформе Vercel с автоматическим деплоем.



Спасибо за внимание!